

메리츠 운영 이관 완료 후 검증 및 연결 설정 가이드

프로그램과 데이터 복사가 완료된 운영 서버에서 연결 설정과 정상 동작을 확인하는 절차

적용 기준 2026 년 9 월 10 일 1 차 이관 실습 결과

보안 원칙 비밀번호 인증 키 토큰은 화면 또는 문서에 기록하지 않는다

중요 이 문서는 재복사나 클러스터 재생성을 기본 절차로 다루지 않는다. 운영 환경의 실제 IP 계정 포트와 서비스 배치가 승인된 구성표와 일치하는지 먼저 확인한다.

1 문서 범위와 작업 순서

운영 이관에서는 프로그램 디렉터리와 데이터가 대상 서버에 이미 복사되어 있다고 가정한다. 따라서 작업자는 다시 복사하기보다 복사 결과를 확인하고, 운영 주소에 맞게 노드별 설정을 수정한 뒤, 하위 인프라부터 상위 애플리케이션 순서로 연결을 검증한다.

단계	작업	완료 기준
1	서버 식별과 파일 확인	hostname IP 디렉터리 소유권이 승인표와 일치
2	이름 해석과 방화벽	서비스 이름이 운영 IP 를 반환하고 필수 포트 통신 가능
3	데이터 계층 확인	MariaDB Redis ZooKeeper Solr Elasticsearch 정상
4	애플리케이션 설정	DB Redis Solr Elasticsearch 내부 API 주소가 운영값
5	순차 기동	프로세스와 LISTEN 포트 정상
6	웹과 업무 기능 검증	로그인 조회 대화 학습 배포가 승인 기준 충족

금지 사항: 확인 없이 Redis cluster create, FLUSHDB, MariaDB 계정 DDL, Solr RESTORE, Elasticsearch restore 를 다시 실행하지 않는다. 이미 복사된 운영 데이터가 있으면 중복 생성 또는 덮어쓰기가 발생할 수 있다.

운영값 기록표

구분	노드 1	노드 2	노드 3	비고
호스트명				승인된 FQDN

구분	노드 1	노드 2	노드 3	비고
운영 IP				실습값 81 82 83 으로 대체 금지
MariaDB 역할	Master	Replica	Replica	실제 역할 확인
Redis 별칭	redis1	redis2	redis3	
ZooKeeper myid	1	2	3	
Solr 별칭	solr1	solr2	solr3	
Elasticsearch 별칭	es1	es2	es3	메리츠 7.8.0

2 공통 사전 확인과 연결 기반

서버와 복사 결과 확인

```
hostname
hostname -i
date '+%F %T %Z'
df -hT / /data /application /tmp 2>/dev/null
ls -ld /application /logs /data 2>/dev/null
```

정상 기준: 접속 서버가 작업 대상과 일치하고, 프로그램과 데이터 경로에 충분한 여유 공간이 있으며, 실행 계정이 필요한 디렉터리를 읽고 쓸 수 있다.

```
for d in mariadb redis solr elk gateway engine master cms chat-ui scheduler; do
[ -e "/application/$d" ] && ls -ld "/application/$d" || echo "$d MISSING"
done
```

MISSING 이 반드시 오류는 아니다. 승인된 서비스 배치표에 해당 서버에 있어야 하는 서비스인지 대조한다.

이름 해석

호스트명은 두 종류를 함께 사용한다. 번호가 붙은 이름은 실제 노드, 번호가 없는 이름은 애플리케이션이 사용하는 논리 주소다. 번호 이름을 지우거나 번호 없는 이름만 세 노드에 중복 등록하면 장애 원인 추적과 장애조치가 어려워진다.

구분	예	용도	가리킬 대상
논리 이름	engine gateway cms master chat-ui	애플리케이션 설정과 브라우저 진입	승인된 VIP 또는 HAProxy
노드 이름	engine1~3 gateway1~3 cms1~3	HAProxy backend 와 노드 직접 점검	각 물리 노드 IP

구분	예	용도	가리킬 대상
인프라 이름	mariadb redis1~3 zk1~3 solr1~3 es1~3	데이터 계층 연결	승인된 서비스 IP

```
getent hosts engine gateway cms master chat-ui
getent hosts engine1 engine2 engine3 gateway1 gateway2 gateway3
getent hosts cms1 cms2 cms3 master1 master2 master3 chat-ui1 chat-ui2 chat-ui3
getent hosts mariadb redis1 redis2 redis3 zk1 zk2 zk3 solr1 solr2 solr3 es1 es2 es3
```

정상 기준: 모든 이름이 승인된 운영 IP 를 반환한다. 실습에서는 무번호 이름을 192.168.1.81 의 HAProxy 로 보냈지만, 운영에서는 승인된 VIP 또는 외부 로드밸런서를 사용한다. source xen02 와 target 운영계가 동시에 켜져 있을 때 동일한 별칭을 양쪽 IP 에 중복 등록하지 않는다.

설정 위치: 서버는 /etc/hosts 또는 운영 DNS 를 사용한다. 작업자 PC 도 CMS 와 시뮬레이터가 실제로 참조하는 cms, chat-ui, gateway 를 해석할 수 있어야 한다. Windows hosts 변경 후 ipconfig /flushdns 를 실행한다.

```
# 실습 예시이며 운영에서는 승인된 VIP 와 실제 노드 IP 로 교체
192.168.1.81 engine gateway cms master chat-ui
192.168.1.81 engine1 gateway1 cms1 master1 chat-ui1
192.168.1.82 engine2 gateway2 cms2 master2 chat-ui2
192.168.1.83 engine3 gateway3 cms3 master3 chat-ui3

# Windows 작업자 PC 예시
192.168.1.81 cms chat-ui gateway engine master
```

포트와 방화벽

서비스	포트	확인 대상
HAProxy	80	작업자 PC 와 내부 사용자
MaxScale	3306	애플리케이션
MariaDB	13306	MaxScale 과 복제 노드
Redis	7000 7001 17000 17001	3 개 Redis 노드 상호 간
ZooKeeper	2181 2888 3888	3 개 ZK 노드와 Solr
SolrCloud	8983	애플리케이션과 Solr 노드 간
메리츠 Elasticsearch	9210 9310	클라이언트와 ES 노드 간
Applications	8081 8180 8280 8380 8480 8580	HAProxy 와 내부 API

```
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --zone=public --list-ports
sudo firewall-cmd --zone=public --list-rich-rules
sudo ss -lntp
```

```
# 노드 간 Engine 통신 예시
for h in engine1 engine2 engine3; do nc -zvw3 "$h" 8180; done
```

애플리케이션 프로세스가 떠 있어도 노드 간 8180 이 막혀 있으면 CMS 학습 배포가 일부 Engine 에 전달되지 않는다. 운영 보안정책에 따라 HAProxy/VIP 와 애플리케이션 노드 사이의 8081, 8180, 8280, 8380, 8480, 8580 을 필요한 방향으로 허용한다. 전체 공개보다 승인된 애플리케이션 서버넷 또는 노드 IP 를 source 로 제한한다.

```
# 예: 승인된 애플리케이션 대역에서 Engine 8180 허용
sudo firewall-cmd --permanent --zone=public \
  --add-rich-rule='rule family="ipv4" source address="<APP_SUBNET>" port port="8180" protocol="tcp" accept'
sudo firewall-cmd --reload
for h in engine1 engine2 engine3; do nc -zvw3 "$h" 8180; done
```

No route to host 또는 TIMEOUT 은 방화벽이나 경로 문제일 가능성이 높고, Connection refused 는 주소에는 도달했지만 서비스가 LISTEN 하지 않는 상태다. ping 성공만으로 TCP 와 HTTP 연결 성공을 판단하지 않는다.

3 MariaDB 와 MaxScale

확인할 것

- 세 노드의 server id 가 서로 다르고 역할이 승인된 구성과 일치하는지 확인한다.
- Replica 두 대의 IO 와 SQL thread, 지연, 마지막 SQL 오류를 확인한다.
- MaxScale 서비스 계정은 사용자명과 접속 Host 조합이 실제 MaxScale IP 와 일치해야 한다.
- 애플리케이션은 개별 MariaDB 가 아니라 MaxScale 별칭 mariadb 와 3306 을 사용한다.

노드와 복제 검증

```
/mariadb/bin/mariadb -S /mariadb/run/mariadb.sock -e \
"SELECT @@hostname, @@server_id, @@read_only; SHOW SLAVE STATUS\G" | \
grep -E
'hostname|server_id|read_only|Master_Host|Slave_IO_Running|Slave_SQL_Running|Seconds_Behind_Master|Last_SQL_Error|Gtid_IO_Pos'
```

정상 기준: Replica 에서 Slave IO Running 과 Slave SQL Running 이 Yes, Seconds Behind Master 가 0, Last SQL Error 가 빈 값이다. Master 에는 SHOW SLAVE STATUS 결과가 없을 수 있다.

계정과 MaxScale 연결

```
/mariadb/bin/mariadb -S /mariadb/run/mariadb.sock -e \
"SELECT User, Host, plugin FROM mysql.user WHERE User='mxs_service';"

sudo maxctrl list servers
nc -zvw3 mariadb 3306
```

설정할 것: 계정 Host 는 MaxScale 이 DB 에 접속할 때 보이는 실제 IP 로 생성한다. MaxScale 서비스의 user 와 password 는 DB 계정과 동일하게 설정한다. 비밀번호는 명령 이력에 직접 남기지 않는 방식을 사용한다.

정상 기준: MaxScale 에서 1 대는 Master Running, 2 대는 Slave Running 이고 애플리케이션 서버에서 mariadb 3306 연결이 된다.

중지 조건: Last SQL Error, GTID out of order, Slave SQL Running No 가 있으면 애플리케이션 기동 전에 복제를 먼저 복구한다.

4 Redis Cluster

설정할 것

항목	노드별 값	주의점
cluster announce ip	redis1 redis2 redis3	각 서버 자신을 가리켜야 함
client ports	7000 7001	두 프로세스 모두 실행
bus ports	17000 17001	노드 상호 통신 허용
data dirs	/application/redis/db/7000 및 7001	admin 쓰기 권한
log dir	/logs/redis	admin 쓰기 권한

```
grep -E '(port|dir|cluster-enabled|cluster-config-file|cluster-announce-ip|cluster-announce-port|cluster-announce-bus-port)' \
/application/redis/conf/7000.conf /application/redis/conf/7001.conf
/application/redis/status.sh
```

클러스터 연결 검증

```
for h in redis1 redis2 redis3; do
  for p in 7000 7001 17000 17001; do nc -zvw2 "$h" "$p"; done
done

/application/redis/bin/redis-cli --cluster check redis1:7000
/application/redis/bin/redis-cli -h redis1 -p 7000 cluster info
/application/redis/bin/redis-cli -h redis1 -p 7000 cluster nodes
```

정상 기준: cluster state ok, known nodes 6, cluster size 3, 16384 slots covered, master 3 대와 replica 3 대가 connected 상태다.

운영 판단: 복사된 nodes.conf 와 데이터가 유효하면 cluster create 를 다시 실행하지 않는다. 노드 주소가 source IP 를 계속 광고하면 복사 방식과 재구성 계획을 별도 승인받는다. Redis cache 는 재생성 정책이면 key 수 불일치만으로 장애로 판단하지 않는다.

5 ZooKeeper

설정할 것

```
grep -Ev '^[[:space:]]*(#|$)' /application/solr/zk/conf/zoo.cfg
cat /application/solr/zk/data/myid
ls -ld /application/solr/zk/data /application/solr/zk/data/log /logs/solr/zk
```

노드	myid	server 항목
zk1	1	server.1=zk1:2888:3888

노드	myid	server 항목
zk2	2	server.2=zk2:2888:3888
zk3	3	server.3=zk3:2888:3888

dataDir, dataLogDir, 로그 디렉터리는 실행 계정 admin 이 쓸 수 있어야 한다. myid 는 각 서버마다 달라야 하고 zoo.cfg 의 server 번호와 일치해야 한다.

연결과 데이터 검증

```
for h in zk1 zk2 zk3; do
  echo "===== $h ====="
  printf 'ruok' | nc -w 3 "$h" 2181; echo
  printf 'srvr' | nc -w 3 "$h" 2181 | grep -E 'Mode|Node count'
done
```

nc 가 없으면 bash 의 /dev/tcp 를 사용한다.

```
timeout 3 bash -c 'exec 3<>/dev/tcp/zk1/2181; printf "srvr" >&3; cat <&3' | grep -E 'Mode|Node count'
```

정상 기준: 세 노드 모두 imok, leader 1 대, follower 2 대, Node count 가 동일하다. 데이터가 복사된 운영 환경에서는 source 와 target 의 znode 또는 Solr chroot 구성이 승인된 결과와 일치하는지도 확인한다.

6 SolrCloud

노드별 연결 설정

```
grep -nE '^(SOLR_HOST|SOLR_PORT|ZK_HOST)' /application/solr/solr/bin/solr.in.sh
```

노드	SOLR HOST	SOLR PORT	ZK HOST
노드 1	solr1	8983	zk1:2181,zk2:2181,zk3:2181/운영 chroot
노드 2	solr2	8983	동일
노드 3	solr3	8983	동일

설정할 것: 세 노드의 SOLR HOST 는 각각 달라야 하며 ZK HOST 는 동일해야 한다. source 와 target 이 같은 ZooKeeper 를 공유하는 구조라면 target 전용 chroot 를 사용한다. 실습에서는 solr-xen01 을 사용했다.

프로세스와 클러스터 검증

```
for ip in <운영_IP_1> <운영_IP_2> <운영_IP_3>; do
  curl -sS --max-time 10 "http://$ip:8983/solr/admin/info/system?wt=json" | \
  grep -E '"mode"|"zkHost"|"solr-spec-version"'
done

curl -sS 'http://solr1:8983/solr/admin/collections?action=LIST&wt=json'
curl -sS 'http://solr1:8983/solr/admin/collections?action=LIST&wt=json'
curl -sS 'http://solr1:8983/solr/admin/collections?action=CLUSTERSTATUS&wt=json' | \
grep -oE '"state": "[^"]+"' | sort | uniq -c
```

정상 기준: 세 노드 모두 mode solrcloud, 동일한 운영 ZK chroot, live nodes 3 대, 필요한 collection 과 alias 존재, replica 가 모두 active 다.

복사 데이터 확인

```
for c in TC_CALLBOT_LOG TC_CALLBOT_B tc_01_A TC_CALLBOT_TEST tc_01_B TC_CALLBOT_A; do
  printf '%-24s ' "$c"
  curl -sS --max-time 30 "http://solr1:8983/solr/$c/select?q=*&rows=0&wt=json" | \
  grep -oE '"numFound": [0-9]+'
done
```

정상 기준: 승인된 source 기준 문서 수와 일치한다. alias TC CALLBOT 과 tc 01 이 각각 올바른 A 또는 B collection 을 가리키는지도 확인한다. 복사된 데이터가 정상인데 collection 을 다시 RESTORE 하지 않는다.

7 Elasticsearch

운영 구조

메리츠용 Elasticsearch 7.8.0 과 기존 Elasticsearch 9.1.1 을 같은 서버에서 동시에 유지하는 경우에만 포트 데이터 로그 경로를 완전히 분리한다. 1 차 실습에서는 충돌 회피를 위해 메리츠 7.8.0 을 9210/9310 으로 변경했다. 실제 개발계에 충돌이 없다면 승인된 원래 포트 9200/9300 을 사용하고 애플리케이션 설정도 그 값에 맞춘다.

구분	HTTP	Transport	data	logs
기존 9.1.1	9200	9300	/var/lib/elasticsearch	/var/log/elasticsearch
메리츠 7.8.0	9210	9310	/application/elk-meritz/data	/logs/elk-meritz

노드별 설정

```
grep -nE '^(cluster.name|node.name|path.data|path.logs|path.repo|network.host|http.port|transport.port|discovery.seed_hosts)' \
/application/elk/config/elasticsearch.yml

grep -nE 'HeapDumpPath|ErrorFile|Xloggc|Xlog:gc' /application/elk/config/jvm.options
```

설정할 것: cluster name 은 trusted-context-es, node name 은 es1 es2 es3, network host 는 각 운영 IP, discovery seed hosts 는 es1:9310 부터 es3:9310 까지다. JVM 로그 경로도 /logs/elk-meritz 로 분리한다. path.repo 는 snapshot 을 사용할 때만 모든 노드에 동일하게 적용한다.

동시 운영과 클러스터 검증

```
sudo ss -lntp | grep -E ':(9200|9300|9210|9310)\b'

curl -sS 'http://es1:9210/_cat/nodes?v&h=name,ip,role,master,version'
curl -sS 'http://es1:9210/_cluster/health?pretty'
curl -sS 'http://es1:9210/_cat/indices?expand_wildcards=all&v&h=health,status,index,docs.count,store.size&s=index'

# 기존 9.1.1 을 유지하는 노드에서 별도 확인
curl -sS 'http://127.0.0.1:9200/' | grep -E '"name"|"cluster_name"|"number"'
```

정상 기준: 메리츠 클러스터는 es1 부터 es3 까지 3 nodes, version 7.8.0, health green, 인덱스와 문서 수가 source 기준과 일치한다. 기존 9.1.1 도 원래 클러스터에 연결되어 있어야 한다.

주의: network host 가 서버 IP 로 지정되면 127.0.0.1 요청은 거부될 수 있다. 이 경우 실제 서버 IP 또는 es1 별칭으로 확인한다.

8 애플리케이션 연결 설정

배치 원칙

gateway engine master cms chat-ui 가 여러 노드에 복사되어 있어도 실제 운영 기동 수와 HAProxy backend 는 승인된 배치표를 따른다. scheduler 는 중복 작업 방지를 위해 승인된 단일 노드에서만 기동하는 것을 기본으로 한다.

반드시 확인하고 수정할 연결값

의존 서비스	권장 연결값	주요 확인 위치	정상 기준
MariaDB	mariadb:3306	runtime.env application.yml	MaxScale 경유 접속
Redis	redis1:7000 redis2:7000 redis3:7000	application.yml	cluster state ok

의존 서비스	권장 연결값	주요 확인 위치	정상 기준
ZooKeeper	zk1:2181 zk2:2181 zk3:2181	Solr 설정	3 대 quorum
Solr	solr1:8983 solr2:8983 solr3:8983	application.yml 또는 내부 설정	운영 chroot 의 collection
Elasticsearch	es1 es2 es3 port 9210	gateway master cms scheduler	trusted-context-es 7.8.0
Master API	http://master	master cms 설정	DNS 와 8380 backend 정상
Engine	논리 주소 http://engine, backend engine1~3:8180	gateway master cms	세 backend 모두 통신 가능
Scheduler	scheduler1:8580/scheduler	batch URL	한 노드만 기동

```
grep -RniE
'mariadb|3306|redis[123]|7000|zk[123]|2181|solr[123]|8983|elasticsearch|es[123]|9200|9210|master|engine[123]|scheduler1' \
/application/{gateway,engine,master,cms,chat-ui,scheduler}/conf 2>/dev/null | \
grep -vE 'before-|backup/'
```

설정할 것: 각 연결값은 실제 운영 배치표를 기준으로 한다. 9210 은 기존 9.1.1 과 충돌했던 실습 전용 값이며, 운영에서 충돌이 없으면 메리츠 7.8.0 의 승인 포트인 9200 을 유지한다. 기존 9.1.1 을 사용하는 별도 애플리케이션 설정은 변경하지 않는다. DB 계정과 비밀번호는 source 의 승인된 운영값을 안전한 방식으로 반영한다.

로그 디렉터리와 기동 전 연결 확인

```
sudo install -d -o admin -g admin -m 750 \
/logs/master /logs/cms /logs/engine /logs/gateway /logs/chat-ui /logs/scheduler

nc -zvw2 mariadb 3306
/application/redis/bin/redis-cli -h redis1 -p 7000 cluster info | grep cluster_state
printf 'ruok' | nc -w 3 zk1 2181; echo
curl -sS --max-time 10 'http://solr1:8983/solr/admin/info/system?wt=json' | grep '"mode"'
curl -sS --max-time 10 'http://es1:<ES_HTTP_PORT>/' | grep -E '"cluster_name"|"number"'
```

다섯 의존 서비스가 모두 정상일 때 애플리케이션을 기동한다. 연결 실패 상태에서 애플리케이션부터 반복 재시작하면 원인 로그가 누적되어 판단이 어려워진다.

9 애플리케이션과 HAProxy 기동

기동 순서

순서	서비스	확인 포트	설명
1	Master	8380	내부 관리 API 선행
2	CMS	8280	Master 연결과 DB 초기화 확인
3	Engine	8180	대화 처리 노드

순서	서비스	확인 포트	설명
4	Gateway	8081	외부 요청 진입
5	Chat UI	8480	사용자 화면
6	Scheduler	8580	승인된 단일 노드만
7	HAProxy	80	backend 확인 후 전환

```
/application/master/start.sh
/application/cms/start.sh
/application/engine/start.sh
/application/gateway/start.sh
/application/chat-ui/start.sh
/application/scheduler/start.sh
```

기동 후 상태 확인

```
for d in master cms engine gateway chat-ui scheduler; do
  echo "==== $d ====="
  /application/$d/status.sh 2>&1
done

sudo ss -lntp | grep -E ':(8081|8180|8280|8380|8480|8580)\b'
```

정상 기준: status 가 running 이고 각 프로세스의 LISTEN 포트가 확인된다. start 출력만 보고 성공으로 판정하지 않는다. 몇 초 후 stopped 가 되면 해당 서비스의 최신 로그를 확인한다.

HAProxy 와 웹 주소

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo systemctl restart haproxy
sudo systemctl --no-pager --full status haproxy
sudo ss -lntp | grep ':80\b'

curl -sS -o /tmp/cms_login.html -w 'HTTP_CODE=%{http_code} SIZE=%{size_download}\n' \
-H 'Host: cms' http://127.0.0.1/login
```

정상 기준: 설정 파일 valid, HAProxy active, 80 LISTEN, CMS login HTTP 200 이다. HTTP 503 은 HAProxy 가 살아 있지만 사용 가능한 backend 를 찾지 못했다는 뜻이다.

작업자 PC 설정: cms 와 meritzeasy cms 뿐 아니라 시뮬레이터가 여는 chat-ui 와 API 호출 대상 gateway 도 운영 HAProxy 또는 VIP IP 로 해석되어야 한다. DNS cache 를 비운 뒤 curl 또는 브라우저로 TCP 80 과 HTTP 응답을 확인한다. ping 성공은 ICMP 확인일 뿐이다.

```
# Windows 관리자 CMD 또는 PowerShell
ipconfig /flushdns
ping cms
ping chat-ui
ping gateway
curl.exe -v --max-time 10 http://gateway/gateway/
```

10 서비스별 최종 판정표

확인	서비스	필수 판정 기준	결과와 시간
[]	MariaDB	복제 IO SQL Yes 지연 0 오류 없음	
[]	MaxScale	1 Master 2 Slave Running mariadb 3306 연결	
[]	Redis	6 nodes 3 masters 3 replicas 16384 slots	
[]	ZooKeeper	3 imok leader 1 follower 2 Node count 동일	
[]	Solr	3 live nodes collection alias replica 문서 수 정상	
[]	ES 7.8	3 nodes green indices 와 문서 수 일치	
[]	ES 9.1	기존 9200 9300 서비스 유지	
[]	Applications	승인된 노드의 프로세스와 포트 정상	
[]	HAProxy	80 LISTEN CMS login HTTP 200	
[]	학습 배포	모든 Engine 통지 성공과 Solr 반영	
[]	시뮬레이터	sessionKey 생성 후 첫 메시지와 왕복 대화	

11 업무 기능 검증

인프라와 포트가 정상이어도 업무 기능은 실패할 수 있다. 완료 판정은 반드시 CMS 화면, API 응답 본문, CMS Gateway Engine 로그를 같은 시간대로 확인한다. HTTP 200 만으로 성공 처리하지 않는다.

확인	기능	검증 방법	정상 기준	증적
[]	CMS 로그인	브라우저에서 /login 접속과 로그인	화면 표시와 로그인 성공	
[]	봇 조회	운영 tenant 의 봇 목록 조회	기존 데이터 표시	
[]	대화 시뮬레이션	대표 시나리오 질의	정상 응답과 오류 없음	

확인	기능	검증 방법	정상 기준	증적
[]	Solr 검색	alias 를 통한 대표 검색	기대 문서 반환	
[]	Redis cache	기능 수행 후 cache key 와 TTL 확인	생성 조회 정상	
[]	학습 배포	테스트 봇으로 학습과 배포	상태 완료와 alias 전환	
[]	Scheduler	예약 배포 또는 허용된 테스트	중복 실행 없이 성공	

학습 배포 필수 확인

승인된 정상 테스트 봇으로 CMS 에서 학습 배포를 실행한다. CMS 가 engine1, engine2, engine3 모두에 업데이트를 통지하고 각 Engine 이 학습 완료 로그를 남기는지 확인한다. 1 차 실습에서는 8180 방화벽 누락으로 engine2 와 engine3 통지가 timeout 되었고, 포트 허용 후 세 노드 모두 성공했다.

```
for h in engine1 engine2 engine3; do
  echo "===== $h:8180 ====="
  nc -zvw3 "$h" 8180
done

# 학습 실행 직후 같은 시간대 확인
grep -RniE 'Succeed notify Engine Update Dialog|Completed Learning Engine|ERROR|Exception|timeout|No route' \
/logs/cms /logs/engine 2>/dev/null | tail -100

curl -sS 'http://solr1:8983/solr/admin/collections?action=LISTALIAS&wt=json' | python3 -m json.tool
```

정상 기준: CMS 에서 배포 완료, 대상 Engine 전부 통지 성공, Engine 에서 STANDBY/TEST 등 승인된 profile 학습 완료, Solr collection 또는 alias 가 기대 상태다. B2B_BASIC_CODE 처럼 source 에서도 channel/intent/alias 가 없었던 봇은 이관 판정용으로 사용하지 않는다.

시뮬레이터 필수 확인

브라우저에서 chat-ui 가 열리는 것만으로는 부족하다. Chat UI 가 gateway 를 통해 세션을 만들고, 응답 JSON 에 null 이 아닌 sessionKey 가 있어야 한다. 200 OK 이면서 body 가 비어 있으면 애플리케이션 관점에서는 실패다.

```
# 운영에서 승인된 정상 botCode 와 channelId 로 교체
curl -sS --max-time 15 \
-D /tmp/simulator.headers -o /tmp/simulator.body \
-w 'HTTP={http_code} SIZE={size_download}\n' \
-X POST -H 'Content-Type: application/json' \
http://gateway/gateway/web/v1/simulator/STANDBY/start \
-d '{"userKey": "<TEST_USER>", "callForwardingNumber": "<TEST_NUMBER>", "channelId": "<APPROVED_CHANNEL_ID>", "reEntry": false}'

grep -oE '"sessionKey"[[[:space:]]*:[[:space:]]*"([^"]+)"' /tmp/simulator.body
```

1 차 실습의 TC_CALLBOT 정상 요청은 code 0000, available true, sessionKey gw1_...를 반환했고 브라우저 대화도 동작했다. 반면 B2B_BASIC_CODE 는 source 와 target 모두 200 + 빈 body 와 sessionKey null 이어서 기존 봇 데이터 문제로 분류했다. 외부 폰트 CORS 오류는 화면 글꼴 문제이며 세션 생성 실패의 원인이 아니었다.

```
for d in master cms engine gateway chat-ui scheduler; do
  echo "===== $d recent errors ====="
  grep -Ei 'ERROR|Exception|failed|timeout|refused' "/logs/$d"/*.log 2>/dev/null | tail -20
done
```

12 최종 일괄 확인 명령

다음 명령은 빠른 상태 요약용이다. 운영 IP 와 실제 경로를 먼저 확인한 뒤 실행한다.

```
echo '==== Names ===='  
getent hosts engine gateway cms master chat-ui engine1 engine2 engine3  
  
echo '==== MariaDB MaxScale ==== '  
sudo maxctrl list servers  
  
echo '==== Redis ==== '  
/application/redis/bin/redis-cli -h redis1 -p 7000 cluster info | \  
  grep -E 'cluster_state|cluster_slots_assigned|cluster_known_nodes'  
  
echo '==== ZooKeeper ==== '  
for h in zk1 zk2 zk3; do printf '%-4s ' "$h"; printf 'ruok' | nc -w 3 "$h" 2181; echo; done  
  
echo '==== Solr ==== '  
curl -sS 'http://solr1:8983/solr/admin/collections?action=LIST&wt=json'  
curl -sS 'http://solr1:8983/solr/admin/collections?action=CLUSTERSTATUS&wt=json' | \  
  grep -oE '"state": "[^"]+"' | sort | uniq -c  
  
echo '==== Elasticsearch ==== '  
curl -sS 'http://es1:<ES_HTTP_PORT>/_cat/nodes?v&h=name,ip,version,master'  
curl -sS 'http://es1:<ES_HTTP_PORT>/_cluster/health?filter_path=status,number_of_nodes,unassigned_shards'  
  
echo '==== Applications ==== '  
for d in master cms engine gateway chat-ui scheduler; do /application/$d/status.sh 2>&1; done  
for h in engine1 engine2 engine3; do nc -zvw3 "$h" 8180; done  
  
echo '==== Web ==== '  
curl -sS -o /dev/null -w 'CMS_LOGIN_HTTP={http_code}\n' -H 'Host: cms' http://127.0.0.1/login
```

13 이상 발견 시 판단 순서

증상	먼저 확인	다음 조치
No route to host	방화벽 route 대상 IP	포트 개방 후 재검증
Connection refused	서비스 프로세스와 LISTEN	로그 확인 후 서비스 기동
STARTED 후 stopped	로그 디렉터리 data 권한	소유권과 실제 오류 수정
HTTP 500	애플리케이션 초기화 로그와 의존 서비스	원인 서비스부터 복구
HAProxy 503	backend 이름 IP 포트 health	backend 기동 또는 HAProxy 설정 수정
브라우저 이름 해석 실패	PC 의 cms chat-ui gateway DNS 또는 hosts	VIP/LB 로 등록 후 DNS cache 제거
Master API 403	인증 없는 직접 호출인지	서비스 장애와 인증 거부를 구분
Solr replica down	노드 간 8983 통신화 core 로그	방화벽 수정 후 recovery 확인
ES path repo empty	세 노드 active settings	모든 노드 설정과 재기동 확인
학습 배포 timeout	engine1~3 이름 8180 방화벽	모든 Engine 연결 후 재배포
시뮬레이터 session null	Gateway start 응답 body 와 channelId	정상 봇 채널로 재검증
HTTP 200 + 빈 body	응답 크기 Gateway Engine 로그	성공으로 판정하지 않고 데이터 확인

Go 조건: 모든 필수 서비스가 정상이고 source 와 target 의 승인된 데이터 기준이 일치하며, 정상 테스트 봇의 학습 배포가 모든 Engine 에서 완료되고 시뮬레이터가 sessionKey 생성 후 실제 대화를 수행한다.

No Go 조건: DB 복제 오류, Redis slot 불완전, ZooKeeper quorum 미구성, Solr down replica, Elasticsearch red, 주요 애플리케이션 반복 종료, CMS login 실패, 학습 배포 실패, 시뮬레이터 session null 중 하나라도 해결되지 않았다.

작업 일시, 운영 IP 와 VIP, 변경한 설정 파일, 검증 증적, 미해결 예외, Go No Go 결정은 별도 핵심 체크리스트에 기록한다.